

Project Overview and Technical Analysis

Agent-Augmented VLA for Bimanual Cloth Manipulation

Client organization: Neotix

Sponsor contact: Birhanu Shimelis Girma

Proposed duration: 14 weeks

Application: Robotic towel folding using a two-arm robot

Primary objective: Improve the performance of a pretrained vision-language-action policy through runtime supervision and correction, without retraining the policy.

1. Executive Summary

This project investigates whether an AI agent can improve the reliability of an existing robotic towel-folding policy by observing its execution, identifying mistakes, and applying corrective interventions while the robot operates.

Neotix will provide a pretrained vision-language-action (VLA) model, a bimanual robot setup, cameras, an inference stack, and an interface for intercepting robot actions. According to the proposal, the existing policy achieves approximately 50–70% end-to-end success in a fixed workspace, with performance declining under changes in lighting, towel characteristics, or initial configurations.

The student team will build a supervisory system around this frozen policy. The supervisor will use a vision-capable language model to analyse observations and proposed actions, determine whether intervention is necessary, and request an approved correction. It will also maintain a memory of previous interventions and their outcomes so that experience from earlier episodes can inform later decisions.

The principal performance target is an improvement of at least 10 percentage points over the frozen policy on real-robot towel folding. Additional evaluation will examine performance on an unseen towel, unfamiliar starting configurations, and an agreed simulation task.

The project combines robotics integration, multimodal reasoning, agent tool design, memory systems, and experimental evaluation. Its feasibility depends strongly on timely access to functioning hardware, a reproducible baseline, and a clearly defined action-intervention interface.

2. Background and Client Context

2.1 Neotix's Existing Capabilities

According to the client proposal, Neotix collects large volumes of human-teleoperated robotics data using bimanual YAM robot stations. Its research team develops learned robot policies, including VLAs, using this data.

For this project, Neotix will provide a checkpoint trained on a few thousand human-teleoperated towel-folding demonstrations. The checkpoint already demonstrates useful task competence but remains inconsistent.

The project therefore begins with an existing robot capability that needs improvement. The students' main responsibility is to develop and evaluate the supervisory system around that capability.

2.2 The Client's Practical Problem

The existing policy can often complete towel folding under familiar conditions, but its performance is vulnerable to variation.

Examples include:

- Changes in lighting.
- Different towel sizes, colours, textures, or thicknesses.
- Unusual towel positions or orientations.
- Partially hidden corners.
- Missed or unstable grasps.
- Incorrect coordination between the two arms.

A local error can cause the rest of the task to fail. For example, if one gripper misses a towel corner but the robot continues as though the grasp succeeded, subsequent lifting and folding movements may be ineffective.

The client wants to investigate whether these errors can be detected and corrected during execution, reducing the need for additional demonstration collection and policy retraining.

2.3 Potential Client Value

If successful, the project could provide:

- Higher task completion rates from an existing policy.
- Recovery from selected execution errors.
- Better performance under some environmental variations.
- Reusable records of failures and successful interventions.

- A supervision framework that can be adapted to other policies or platforms.

These benefits must be assessed alongside additional inference costs, execution delays, and system complexity.

3. Problem Statement

The existing pretrained VLA does not consistently complete towel-folding tasks, particularly when operating conditions differ from those represented in its training data.

The central problem is:

How can a vision-language-model-based supervisor improve the end-to-end success of a frozen robotic policy through timely, bounded interventions and experience retrieval, without changing the policy's learned parameters?

Solving this problem requires more than identifying mistakes. The system must determine whether intervention is justified, translate its decision into an executable correction, apply that correction at an appropriate time, and evaluate the resulting outcome.

4. Project Aim and Objectives

4.1 Overall Aim

Develop and evaluate an agent-based supervisor that improves the reliability of a pretrained bimanual towel-folding policy during inference.

4.2 Specific Objectives

1. **Reproduce the baseline.**
Establish the frozen policy's performance under agreed operating conditions.
2. **Build an observation and intervention harness.**
Capture synchronised observations, record actions, and support approved overrides.
3. **Develop an initial overseer.**
Diagnose selected failure conditions and request bounded corrective actions.
4. **Implement cross-episode memory.**
Store and retrieve previous situations, interventions, and observed outcomes.
5. **Evaluate generalization.**
Test the system on at least one unseen towel and one held-out starting-pose distribution.
6. **Measure component contributions.**
Compare the frozen policy, supervision without memory, and supervision with memory.
7. **Demonstrate portability.**
Apply the supervisory approach to the simulation task agreed with Neotix.

8. Produce a reproducible handover.

Deliver code, documentation, evaluation results, and a demonstration video.

5. Key Technical Concepts

Concept	Meaning within this project
Vision-language-action model	A model that uses visual observations and a language instruction to generate robot actions. The precise inputs depend on the supplied checkpoint.
Policy	The mechanism that maps observations and task information to robot actions.
Frozen policy	A policy whose learned parameters remain unchanged.
Inference time	The period when the trained model is running and generating outputs.
Bimanual manipulation	Coordinated manipulation using two robot arms.
End effector	The device at the end of a robot arm, typically a gripper in this project.
Action chunk	A sequence of future robot commands predicted together.
Rollout or episode	One attempt to complete the task, from an initial state to success, failure, or termination.
Overseer	The agent that assesses execution and chooses whether to request intervention.
Cross-episode memory	Stored experience that remains available after an episode ends and the environment resets.
Zero-shot evaluation	Evaluation without examples or adaptation from the held-out condition beforehand.
Few-shot evaluation	Evaluation after a specified small amount of experience with a new condition.
Ablation	An experiment that removes or changes a component to measure its contribution.

6. Proposed System Architecture

6.1 Main Components

The system will contain the following components:

Component	Responsibility
Observation layer	Collect camera images, timestamps, and available robot-state information.
Frozen VLA	Generate the next proposed action chunk.
Supervisor	Analyse observations, proposed actions, and relevant memory.
Intervention interface	Convert supervisor decisions into structured requests.
Validation and scheduling layer	Check whether a request is permitted, valid, and still timely.
Robot controller	Execute accepted actions through Neotix's control stack.
Outcome assessment	Record whether the intended local correction and overall task succeeded.
Memory store	Preserve intervention experiences for later retrieval.

The proposal explicitly gives the supervisor camera observations, proposed actions, and correction memory. Access to robot state and execution progress should also be discussed with Neotix because these can improve the interpretation and timing of interventions.

6.2 Operating Sequence

A typical episode would proceed as follows:

1. The robot and towel are placed in an agreed initial configuration.
2. The system receives the towel-folding instruction.
3. Cameras capture the workspace.
4. The frozen VLA proposes an action chunk.
5. The supervisor assesses the current situation when scheduled or triggered.
6. The supervisor allows execution or requests an intervention.
7. The validation layer checks the intervention's structure, bounds, and relevance.
8. The controller executes the accepted action.
9. New observations are collected.
10. The intervention and its observed outcome are logged.
11. The process continues until success, failure, or another termination condition.

This sequence describes the logical workflow. In an asynchronous implementation, some observation, reasoning, and execution activities will overlap.

6.3 Illustrative Example

The robot attempts to grasp two towel corners. The left gripper closes without capturing the towel, while the right gripper succeeds.

The supervisor identifies a likely missed grasp and requests an approved recovery procedure. The controller coordinates the recovery so that the other arm does not continue with an incompatible movement.

After execution, the system checks whether the left corner now moves with the gripper. It records the intervention as successful, unsuccessful, or uncertain based on the available evidence.

7. Intervention Mechanisms

7.1 Allow the Action Unchanged

The supervisor permits the VLA's proposed action to proceed.

This is an important decision because excessive intervention can reduce performance. The supervisor must learn, through its design and evaluation, when available evidence is insufficient to justify a correction.

7.2 Apply an End-Effector Offset

The supervisor requests a small adjustment to the intended position of one or both grippers.

Implementation requires agreement on:

- Coordinate frames.
- Units and permitted offset magnitudes.
- Whether orientation changes are allowed.
- How offsets are applied across an action chunk.
- How end-effector commands relate to the VLA's native action representation.
- How the two arms remain coordinated.

A visual observation such as “the gripper is beside the corner” does not directly establish a reliable three-dimensional correction. Camera calibration, geometric tools, or other client-provided capabilities may be required.

7.3 Select Among Candidate Action Chunks

If supported by the policy, the system generates several candidate action chunks and allows the supervisor to select one.

This option depends on:

- Whether candidate sampling is available.
- Whether candidates differ meaningfully.
- The cost of generating additional candidates.
- How proposed movements are represented to the supervisor.
- Whether the supervisor can assess their likely consequences.

The ability to generate and compare candidates should be confirmed before including this feature in the minimum deliverable.

7.4 Request a Short New Plan

The supervisor requests a revised short sequence or recovery subtask.

Possible interpretations include:

- Asking the VLA to replan from updated observations.
- Supplying a revised language instruction.
- Calling a predefined recovery routine.
- Selecting a short sequence of approved tools.

Neotix should clarify which interpretation is intended and what execution support already exists.

7.5 Recommended Initial Scope

The first online version should support pass-through and one bounded correction mechanism aimed at a small number of common failures.

Additional mechanisms can be introduced after the initial system demonstrates reliable execution and measurable benefit.

8. Cross-Episode Memory

8.1 Purpose

Memory allows the supervisor to reuse previous experience without updating the VLA's parameters.

The intended benefit is that similar mistakes can be addressed more effectively in later episodes.

8.2 Suggested Memory Record

Field	Description
Task stage	Where the robot was in the folding process.
Observation reference	Images or logs associated with the situation.
Failure diagnosis	The suspected problem and supporting evidence.
Relevant state	Available gripper positions, execution progress, or other context.
Intervention	The requested correction and parameters.
Local outcome	Whether the immediate problem appeared to improve.
Episode outcome	Whether the full task eventually succeeded.
Applicability conditions	Circumstances under which the experience may be relevant.
Uncertainty	Whether the outcome or diagnosis remains ambiguous.

8.3 Memory Quality

The system should store failed and uncertain interventions as well as successful ones.

It should also distinguish between:

- A correction that improved an immediate grasp.
- A correction followed by eventual task success.
- Evidence that the correction itself caused the improvement.

These are related but different claims.

Memory retrieval should consider context. A fixed directional offset that helped in one configuration may be inappropriate after the towel rotates or the camera viewpoint changes.

8.4 Memory During Evaluation

Memory handling must be specified for every experiment.

For held-out conditions, evaluation should distinguish:

- Initial performance before experience with the new condition.
- Performance after a fixed adaptation budget.
- Performance as additional episodes accumulate.

Allowing memory to grow during testing without reporting that growth would make zero-shot and few-shot results difficult to interpret.

9. Runtime and Latency Design

9.1 The Timing Challenge

The supervisor may take longer to produce a decision than the robot takes to execute the action under review.

For example, if a chunk finishes before the model response arrives, the requested correction may no longer be relevant.

9.2 Asynchronous Supervision

An asynchronous design allows the robot's control processes to continue while the supervisor analyses observations.

The system must then determine whether a returned decision is still applicable.

Recommended mechanisms include:

- Timestamps for observations and requests.
- Action-chunk identifiers.
- Explicit execution progress.
- Permitted intervention points.
- Response-expiration rules.
- A client-approved fallback for invalid or delayed responses.

Depending on the current state, the approved fallback could involve continuing a validated action, holding position, or stopping through the existing controller. This behaviour should be defined with Neotix.

9.3 Latency Metrics

Evaluation should measure:

- Supervisor response time.
- Time from observation to applied correction.
- Frequency of stale responses.
- Added pauses.
- Total episode duration.
- Robot command rate.

Maintaining command frequency alone does not establish acceptable task speed.

10. Data, Hardware, and Client Dependencies

10.1 Resources Promised in the Proposal

Neotix proposes to provide:

1. A pretrained towel-folding VLA checkpoint.
2. One or two bimanual YAM robot stations.
3. One wrist camera per arm and an environment camera.
4. A LeRobot-based inference stack.
5. A Python interface for intercepting and overriding actions.
6. Real cloth items and staged failure scenarios.
7. API funding for a frontier vision-language model.
8. A LIBERO or LIBERO-Plus simulation harness.
9. Weekly engineering meetings and a support channel.

10.2 Dependencies Requiring Confirmation

The team should confirm:

- When each resource will become available.
- Where the robots are located.
- Whether access is local or remote.
- Guaranteed robot availability.
- Who resets the towel between trials.
- Available computing resources for VLA inference.
- Camera calibration and synchronisation status.
- The exact action representation.
- Existing safety and collision-handling capabilities.
- Simulation tasks and compatible policies.
- Code, data, and publication permissions.

These dependencies directly affect what can be delivered within 14 weeks.

11. Project Scope

11.1 In Scope

- Running and evaluating the supplied frozen policy.
- Capturing and analysing robot rollouts.
- Developing a vision-language-model supervisor.
- Implementing agreed intervention mechanisms.
- Building cross-episode memory.

- Measuring success, latency, and intervention behaviour.
- Testing agreed held-out conditions.
- Demonstrating the wrapper in the supplied simulation setup.
- Producing code, documentation, and a final report.

11.2 Outside the Proposed Core Scope Unless Separately Agreed

- Training a new VLA.
- Large-scale collection of additional demonstrations.
- Redesigning robot hardware.
- Building a complete cloth simulator from scratch.
- Developing unrestricted autonomous robot-control code.
- General-purpose handling of all garment types.
- Guaranteeing production-level reliability.

11.3 Minimum Viable Deliverable

A practical minimum deliverable would contain:

- A reproducible baseline.
- An operational logging and intervention harness.
- An overseer addressing selected failure types.
- One approved correction mechanism.
- An outcome-linked memory system.
- Controlled comparisons against the baseline.
- The agreed generalization and simulation tests.

12. Evaluation Plan

12.1 Primary Success Metric

The proposal specifies at least a 10-percentage-point improvement in real-robot towel-folding success.

For example:

Configuration	Successful episodes	Success rate
Frozen policy	18 out of 30	60%
Supervised policy	21 out of 30	70%

This represents a 10-percentage-point increase.

However, a difference of three successes in 30 trials is subject to substantial uncertainty. Trial counts and uncertainty reporting should be agreed before final evaluation.

12.2 Operational Definition of Success

The client and team must define:

- Required fold pattern.
- Acceptable alignment or placement tolerance.
- Maximum task duration.
- Permitted retries.
- Treatment of dropped towels.
- Treatment of human assistance.
- Treatment of safety stops.
- How ambiguous outcomes are labelled.

The same criteria must apply across compared systems.

12.3 Essential Comparisons

Configuration	Purpose
Frozen VLA	Establish baseline performance.
VLA with supervisor, memory disabled	Measure the contribution of immediate supervision.
VLA with supervisor and memory	Measure the additional contribution of stored experience.

If time permits, compare against a simple recovery-rule system and an alternative overseer model.

12.4 Generalization Tests

The proposal requires:

- At least one held-out towel.
- At least one held-out starting-pose distribution.

These conditions should remain separate from development and prompt tuning. Zero-shot and few-shot protocols should be documented independently.

12.5 Secondary Metrics

Recommended secondary metrics include:

- Task completion time.
- Number of interventions per episode.
- Recovery rate for targeted failures.
- Unnecessary or harmful interventions.
- Supervisor latency.
- Stale-response frequency.
- Human assistance and safety stops.
- API calls and cost per episode.

12.6 Experimental Consistency

Use comparable initial conditions, consistent limits, and documented randomisation where appropriate. Interleaving baseline and supervised trials can reduce confounding from changes in hardware or environmental conditions over time.

Report confidence intervals or another agreed uncertainty measure alongside success rates.

13. Simulation Component

The simulation component is intended to demonstrate that the supervisory approach can operate beyond the physical YAM setup.

The proposal names LIBERO and LIBERO-Plus, but the exact task must be clarified. The team should not assume that the supplied harness includes bimanual cloth simulation.

Questions to resolve include:

- What robot and manipulation task are simulated?
- Is a compatible pretrained policy supplied?
- What observations and action interfaces are available?
- Which supervisory components should transfer unchanged?
- Is the simulation intended to test cloth manipulation or software portability?

If the simulated task differs from towel folding, the resulting evidence should be presented as a portability demonstration. It should not be described as direct validation of simulated cloth-folding performance.

14. Research Questions and Contribution

14.1 Main Research Question

Can a vision-language-model-based agent improve the success of a frozen bimanual cloth-manipulation policy through runtime intervention?

14.2 Supporting Questions

1. Which failure types are sufficiently observable and recoverable?
2. When does supervision help, and when does it disrupt execution?
3. Does memory improve performance beyond immediate diagnosis?
4. How does response latency affect intervention usefulness?
5. Do benefits transfer to unseen towels and initial configurations?
6. How much task-specific engineering is needed for a second platform?

14.3 Potential Contribution

The contribution could be a demonstrated method for combining:

- A frozen learned policy.
- Multimodal diagnosis.
- Bounded action interventions.
- Cross-episode experience retrieval.
- Real-robot evaluation on deformable-object manipulation.

The novelty claim remains provisional until relevant literature has been reviewed. The client should provide exact links for the cited ENPIRE and ASPIRE work before the team relies on the proposal's comparison.

15. Risks and Mitigation

Risk	Potential effect	Proposed response
Delayed hardware access	Compresses integration and evaluation time.	Confirm access early and begin with recorded rollouts.
Baseline cannot be reproduced	Makes improvement claims unreliable.	Reproduce and document the baseline before development proceeds far.
Incorrect visual diagnosis	Causes unnecessary or harmful interventions.	Start with clearly observable failure types and evaluate false interventions.
Weak geometric grounding	Produces inaccurate physical corrections.	Use calibrated tools and client-supported coordinate conversions.
Slow supervisor responses	Corrections arrive after the relevant action.	Use asynchronous scheduling, timestamps, and expiry rules.

Two-arm coordination failures	One correction disrupts the other arm.	Validate interventions in the context of the joint task.
Unreliable memory	Repeats ineffective corrections.	Store outcomes and applicability conditions, including failures.
Variable cloth behaviour	Produces noisy performance estimates.	Standardise evaluation and report uncertainty.
Simulation mismatch	Prevents the intended portability demonstration.	Agree on the task, policy, and interface early.
Excessive scope	Leaves limited time for credible evaluation.	Prioritise one intervention mechanism before expanding.

Real-robot operation must use Neotix's required end-effector limits and human emergency stop. Any additional validation requirements should be agreed with its engineers and enforced outside the language model.

16. Semester Work Plan

Period	Main activities	Expected output
Weeks 1–2	Onboarding, hardware access, interface inspection, baseline reproduction, evaluation agreement.	Baseline report and confirmed implementation scope.
Weeks 3–5	Rollout logging, offline diagnosis, initial structured interventions, supervised online testing.	Working overseer version 1.
Weeks 6–8	Failure taxonomy, intervention outcome assessment, memory storage and retrieval.	Supervisor with cross-episode memory.
Weeks 9–11	Held-out tests, robustness analysis, agreed simulation integration.	Generalization and portability results.
Weeks 12–13	Final trials, ablations, latency and cost analysis.	Quantitative evaluation package.
Week 14	Documentation, report, demo video, code handover.	Final project artifact.

Evaluation planning should begin in the first two weeks, even though most final trials occur later.

17. Final Deliverables

1. **Reproducible codebase**
Includes the supervisory loop, intervention tools, configuration files, and execution instructions.
2. **Baseline evaluation report**
Documents the checkpoint, operating conditions, success definition, and reproduced performance.
3. **Rollout and intervention harness**
Captures observations, proposed actions, executed actions, interventions, and outcomes.
4. **Overseer implementation**
Supports structured diagnosis and agreed correction mechanisms.
5. **Cross-episode memory system**
Stores and retrieves intervention experience with outcome information.
6. **Evaluation results**
Covers baseline comparisons, ablations, held-out conditions, latency, and the agreed simulation task.
7. **Final technical report**
Explains the method, implementation, results, limitations, and future work.
8. **Demonstration video**
Shows representative successes, recoveries, and remaining failures.
9. **Handover documentation**
Explains installation, operation, configuration, and known limitations.

18. Questions for the Initial Client Meeting

Technical Interface

1. Which VLA checkpoint will be supplied?
2. What observations and robot-state inputs does it use?
3. Does it output joint commands, end-effector poses, or another action representation?
4. How long are action chunks, and how frequently does the policy replan?
5. What changes does the intervention API support?
6. Are calibration and coordinate-conversion tools already available?
7. Does the policy support multiple candidate action chunks?
8. What does requesting a short new plan mean in the current stack?

Access and Resources

9. Where are the robots, and what access is guaranteed?
10. Who resets and supervises physical trials?
11. What compute and API budget will be available?

12. When will the checkpoint, example rollouts, and simulation harness be delivered?

Evaluation and Scope

- 13. What is the precise definition of successful folding?
- 14. Which conditions underlie the reported 50–70% baseline?
- 15. What does “30 trials per setting” include?
- 16. Which intervention mechanisms are mandatory?
- 17. What latency and task-duration limits are acceptable?
- 18. How will memory be controlled during held-out evaluation?
- 19. Which simulation task and compatible policy are supplied?
- 20. Is the 10-point improvement a research target or a contractual acceptance requirement?

Research and Handover

- 21. Can the client provide the exact related-work references?
- 22. What code and data may be included in the final artifact?
- 23. Are publication or public demonstrations permitted?
- 24. Who will maintain the system after handover?

19. Overall Assessment

This proposal offers a well-defined opportunity to study runtime improvement of learned robot behaviour using real hardware and an existing policy. Its strongest assets are the client-provided checkpoint, robot stations, inference stack, and engineering support.

The most demanding part is converting a visual diagnosis into a correction that is physically valid, coordinated across both arms, and timely enough to help. Memory and prompting are valuable components, but their effectiveness depends on this execution interface.

A manageable project should begin with a reproducible baseline and a narrow set of recoverable failures, demonstrate one intervention mechanism, and then add memory and broader evaluation. The 10-percentage-point improvement should be treated as a measurable research target whose achievement must be established experimentally.